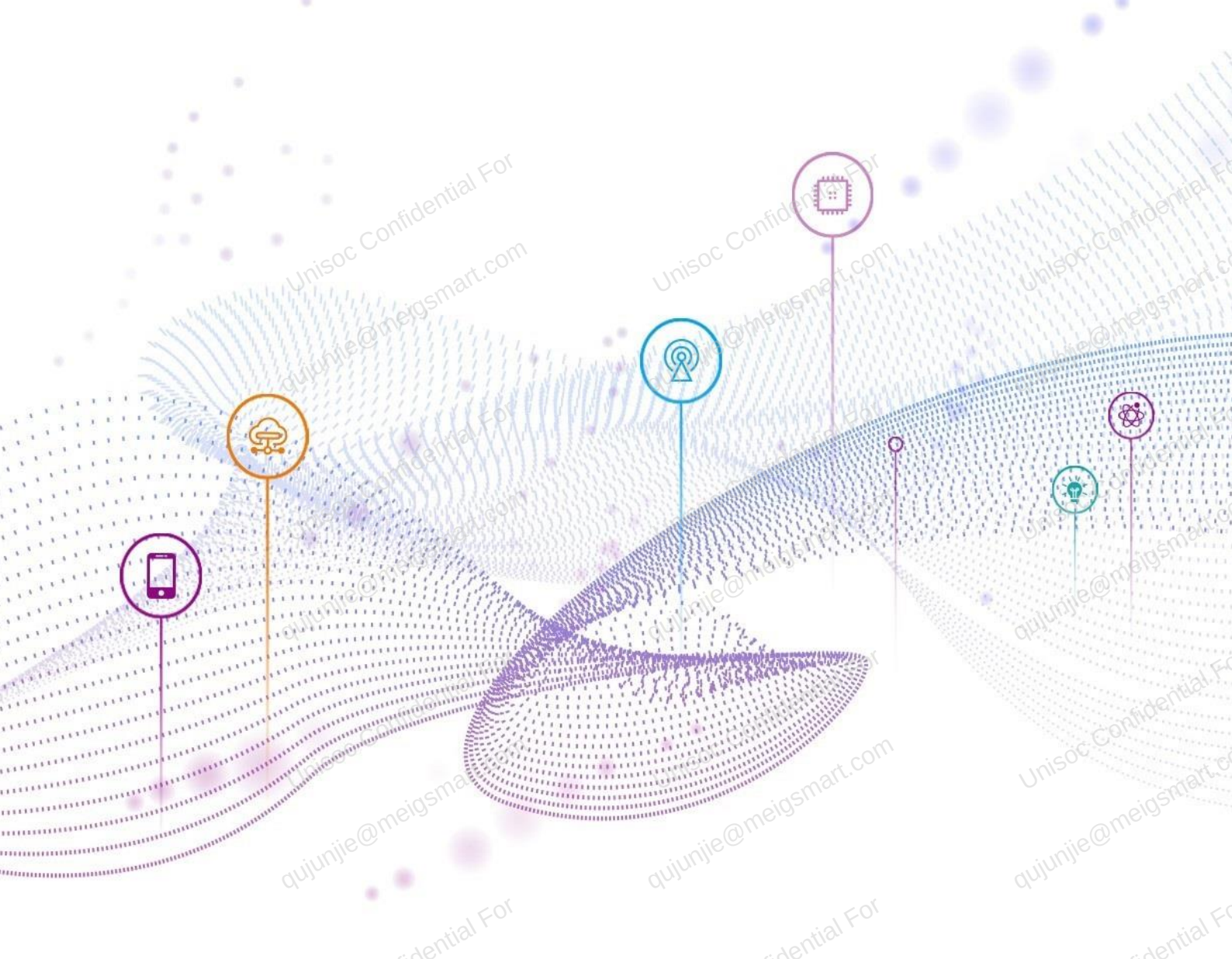




Android 11.0 ~ 12.0 SensorHub 动态加载驱动指导

文档版本
发布日期

V1.3
2022-02-15

版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

紫光展锐（上海）科技有限公司



前言

概述

该文档适用于展锐当前带 SensorHub 平台的产品，主要介绍 SensorHub 模块动态加载机制、动态加载机制系统接口、动态加载机制 API 接口、动态加载驱动范例及编译。

读者对象

此文档供 sensor 模块相关技术人员及展锐客户参考使用。

缩略语

缩略语	英文全名	中文解释
OPCODE	Open Code	在 Sensors HAL 层对 Sensor 进行参数配置的文件。
RTOS	Real Time Operating System	实时操作系统：是指当外界事件或数据产生时，能够接受并以足够快的速度予以处理的操作系统。

变更信息

文档版本	发布日期	修改说明
V1.0	2020-09-28	第一次正式发布。
V1.1	2021-03-05	4.1 章节增加 UMS9230 平台信息。
V1.2	2021-11-12	- 将文档名由《Android 11.0 SensorHub 动态加载驱动指导》修改为《Android 11.0 ~ 12.0 SensorHub 动态加载驱动指导》。 - 新增 2.5 send_dynamic_data_to_ap 接口和 3.7 色温传感器 API。
V1.3	2022-02-15	新增 3.8 dynamic_receive_data 接口 。

关键字

SensorHub、动态加载驱动

目 录

1 动态加载机制介绍.....	1
2 动态加载机制系统接口.....	2
2.1 i2c_read 和 i2c_write 接口	3
2.2 gpio_set 接口.....	5
2.3 msleep 接口.....	5
2.4 LOG_PRINTF 宏接口	6
2.5 send_dynamic_data_to_ap 接口.....	6
3 动态加载机制 API 接口	8
3.1 磁力计 API.....	9
3.1.1 mag_init 接口.....	9
3.1.2 mag_update 接口	10
3.1.3 mag_close 接口	11
3.1.4 mag_update_cfg 接口	11
3.2 加速度计 API.....	12
3.2.1 acc_enable 接口	12
3.2.2 acc_get_data 接口	13
3.2.3 acc_disable 接口	13
3.2.4 acc_update_cfg 接口	14
3.3 陀螺仪 API.....	14
3.3.1 gyro_enable 接口	14
3.3.2 gyro_get_data 接口	15
3.3.3 gyro_disable 接口	16
3.3.4 gyro_update_cfg 接口	16
3.4 压力传感器 API.....	17
3.4.1 pressure_enable 接口	17
3.4.2 pressure_get_data 接口	17
3.4.3 pressure_disable 接口	18
3.4.4 pressure_update_cfg 接口	18
3.5 光线传感器 API.....	19
3.5.1 light_enable 接口	19
3.5.2 light_get_data 接口	20
3.5.3 light_disable 接口	20
3.5.4 light_update_cfg 接口	21
3.6 距离传感器 API.....	21
3.6.1 proximity_enable 接口	21

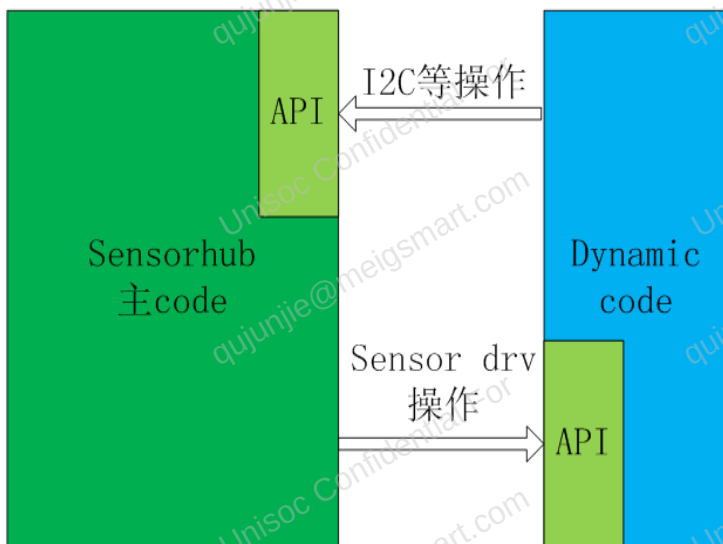
3.6.2 proximity_get_data 接口	22
3.6.3 proximity_disable 接口	23
3.6.4 proximity_update_cfg 接口	23
3.7 色温传感器 API	24
3.7.1 color_enable 接口	24
3.7.2 color_update_cfg 接口	24
3.7.3 color_get_data 接口	25
3.7.4 color_disable 接口	26
3.8 dynamic_receive_data 接口	26
4 动态加载驱动范例及编译	28
4.1 动态加载驱动范例	28
4.2 动态加载驱动编译	28

1 动态加载机制介绍

在实际开发中，有些 sensor 并不是简单的配置一下寄存器就可以正常工作，而需要搭配特定的算法才能正常工作。例如磁力计，其 raw data 需要进行校准才能正常使用，而不同厂家的校准算法各有不同，很难达到统一。又例如距离传感器的快速校准算法，每个厂家都有自己的一套逻辑。针对这种情况，展锐推出动态加载机制，解决各厂家之间的差异问题。

动态加载机制的调用示意图如图 1-1 所示。

图1-1 动态加载机制示意图



动态加载机制和 Sensorhub 主 code 分开编译，两者通过定义好的 API 相互调用，所以只要 Dynamic code 中 API 的地址和定义不变，无论其中内容是什么，对 Sensorhub 主 code 来讲都是透明的。

说明

使用动态加载驱动机制前，请 sensor 驱动开发人员参阅对应平台对应 Android 版本的 SensorHub 客制化指导手册，了解 Sensors HAL 对各类型 sensor 参数的配置以及使用动态加载机制时如何配置相关参数。

2 动态加载机制系统接口

动态加载机制与 SenorHub 的主 code 分开编译，在动态加载机制中无法直接调用系统中的相应接口，所以只能通过回调函数的方式来使用。目前为动态加载机制开放的系统接口如下所示：

```
struct i2c_dev_info {
    /* to specify which i2c to use, i2c0: interface_num = 0, i2c1: interface_num = 1, ... */
    uint32_t interface_num;
    /* i2c slave device's write address */
    uint8_t slave_addr;
    /* 1: for reg address is 1 byte i2c , 2: for 2 bytes reg address */
    uint32_t reg_addr_len;
};

struct shub_sys_interface {
    /****** init_param ---- deprecated *****/
    uint32_t (*init_param)(int32_t *param, int size);
    /****** debug_trace ---- log print interface *****/
    uint32_t (*debug_trace)(const char *x_format, ...);
    /****** extern_i2c_read ---- deprecated *****/
    uint32_t (*extern_i2c_read)(uint32_t handle, uint8_t *reg_addr, uint8_t *buffer, uint32_t bytes);
    /****** extern_i2c_write ---- deprecated *****/
    uint32_t (*extern_i2c_write)(uint32_t handle, uint8_t *reg_addr, uint8_t *buffer, uint32_t bytes);
    /* msleep ---- set cpu to sleep interface, time unit is ms */
    void (*msleep)(uint32_t ms);
    /****** extern_i2c_open ---- deprecated *****/
    int32_t (*extern_i2c_open)(void *i2c_dev);
    /****** extern_i2c_close ---- deprecated *****/
    int32_t (*extern_i2c_close)(uint32_t handle);
    /* extern_gpio_config ---- config gpio voltage level interface */
    void (*extern_gpio_config)(uint32_t gpio_num, bool value);
    /* sensor_i2c_read ---- i2c read interface */
    int32_t (*sensor_i2c_read)(struct i2c_dev_info *i2c_info, uint8_t *reg_addr, uint8_t *buffer, uint32_t bytes);
    /* sensor_i2c_write ---- i2c write interface */
    int32_t (*sensor_i2c_write)(struct i2c_dev_info *i2c_info, uint8_t *reg_addr, uint8_t *buffer, uint32_t
```



```
bytes);
/*sensor_data_to_ap -- send dynamic data to ap, warning: not beyond 30 bytes */
int32_t (*sensor_data_to_ap)(uint8_t type, void *data, uint32_t bytes);
};
```

说明

上述接口中部分接口已经弃用。已弃用的接口注释中有 deprecated 字样。在开发软件中不必再关注此类接口。只是为了兼容老版本软件，保留了声明。

最新版本的代码中对上述接口进行了简单封装，使用时按普通函数接口调用方法使用即可，参见 vendor/sprd/modules/sensors/libsensorhub/dynamic_driver/目录下的 sensorhub_interface.h 和 sensorhub_interface.c 实现，使用这些接口时直接#include “sensorhub_interface.h”即可。

```
int32_t i2c_read(struct i2c_dev_info *i2c_info, uint8_t *reg_addr, uint8_t *reg_value, uint32_t regs_num);
int32_t i2c_write(struct i2c_dev_info *i2c_info, uint8_t *reg_addr, uint8_t *reg_value, uint32_t regs_num);
void gpio_set(uint32_t gpio_num, bool value);
void msleep(uint32_t ms);
#ifdef LOG_PRINTF
extern const volatile struct shub_sys_interface * const shub_extern_call;

#define LOG_PRINTF(fmt, args...) shub_extern_call->debug_trace(fmt, ##args)
#endif
```

上述接口分别用于 SensorHub i2c 读写，GPIO 输出电平配置，系统休眠和 log 打印。

下面对这些接口使用方法进行介绍。

2.1 i2c_read 和 i2c_write 接口

【函数功能】

对 sensor 进行 i2c 读写操作。

【函数原型】

```
int32_t i2c_read(struct i2c_dev_info *i2c_info, uint8_t *reg_addr, uint8_t *reg_value, uint32_t regs_num);
int32_t i2c_write(struct i2c_dev_info *i2c_info, uint8_t *reg_addr, uint8_t *reg_value, uint32_t regs_num);
```

【参数说明】

参数	说明
i2c_info	挂载的 i2c 从设备信息，包括使用哪个 i2c 端口(SensorHub 上目前仅有 i2c0 或 i2c1)，设备写地址，设备寄存器地址占几个字节。
reg_addr	要读写的寄存器地址指针。

参数	说明
reg_value	i2c_read 中为读出寄存器值的指针，i2c_write 中为要写入寄存器的值的指针。 可以为多个值的首地址。
regs_num	需要连续读写的寄存器数量。 - 如果一次仅读写一个寄存器，配置为 1。 - 如果读或写连续的多个寄存器，则配置为要读写寄存器数量。如要连续读 3 个寄存器：0x14, 0x15, 0x16，则配置为 3。

【返回值】

- 成功：返回 regs_num 值。
- 失败：返回一个负值。

【示例】

假设 sensor A 使用 i2c0，i2c 从设备（即 sensor）写地址为 0x92，寄存器地址为 16 bit（一般情况 sensor 寄存器地址为 8 bit，本示例中为 16 bit）。

读写寄存器 0x0123，示例如下。

//sensor i2c info 定义，结构体成员含义见原型注释

```
struct i2c_dev_info a_sensor_i2c_info = {
.interface_num = 0,
.slave_addr = 0x92,
.reg_addr_len = 2 //此示例寄存器地址为16 bit，如果是8bit，reg_addr_len为1
};
```

//读寄存器0x0123，读值保存在reg_value示例：

```
int ret = -1;
uint16_t reg_addr = 0x0123;
uint8_t reg_value = 0xff;
ret = i2c_read(&a_sensor_i2c_info, &reg_addr, &reg_vaule, 1);
//写寄存器0x0123，写入值为0x08示例：
int ret = -1;
uint16_t reg_addr = 0x0123;
uint8_t reg_value = 0x08;
ret = i2c_write(&a_sensor_i2c_info, &reg_addr, &reg_vaule, 1);
```

2.2 gpio_set 接口

【函数功能】

对 GPIO 输出电平进行控制。

【函数原型】

```
void gpio_set(uint32_t gpio_num, bool value);
```

【参数说明】

参数	说明
gpio_num	SensorHub 侧 GPIO 端口号。
value	高电平为 1，低电平为 0。

【返回值】

无。

【示例】

将 SensorHub 侧 GPIO50 设置为高电平：

```
gpio_set(50, 1);
```

2.3 msleep 接口

【函数功能】

休眠接口。

【函数原型】

```
void msleep(uint32_t ms);
```

【参数说明】

参数	说明
ms	设置 CPU 休眠时长，单位毫秒（ms）。

【返回值】

无。

【示例】

让 SensorHub 休眠 10 毫秒。

```
msleep(10);
```

2.4 LOG_PRINTF 宏接口

【函数原型】

```
LOG_PRINTF(fmt, args...)
```

与 C 语言标准库 printf 函数接口使用方法相同，不再赘述。

2.5 send_dynamic_data_to_ap 接口

【函数功能】

将动态加载的数据发送给 AP

【函数原型】

```
int32_t (*sensor_data_to_ap)(uint8_t type, void *data, uint32_t bytes);
```

【参数说明】

参数	说明
type	sensor 类型。
data	动态加载向 AP 发送值的指针。
bytes	data 的大小。

【返回值】

- 成功：返回 1
- 失败：返回 0

【示例】

以光线传感器从动态加载向 AP 传递校准参数 light_cali 为例介绍该接口的使用：

1. 调用该接口进行参数传递：

```
send_dynamic_data_to_ap(LIGHT, &light_cali, sizeof(light_cali));
```

2. AP 侧解析函数为 void get_dynamic_data (struct shub_data *sensor)。通过该函数将动态加载传递来的参数进行解析并打印。如在此例中，将打印出 LIGHT 的 handle 值和 light_cali 的值。

说明

AP 侧的解析函数只是作为此接口的参考示例，客户具体使用过程中可根据自己的需求进行修改。

3 动态加载机制 API 接口

整个动态加载机制可使用的空间只有 20 K，所以能用 opcode 完成的部分尽量不用动态加载机制。目前 SensorHub 一共支持 7 种实体 sensor，每种 sensor 都有四个独立的 API 供开发者使用，这些 API 的定义如下所示。

路径：vendor/sprd/modules/sensors/libsensorhub/dynamic_driver/dynamic_driver.h

```
struct dynamic_driver {
    /* mag sensor extern call */
    int (*mag_init)(int32_t sampleTime_ms, float *mag_offset);
    int (*mag_update)(double *mag, double *acc, double *gyro, float *cali_mag,
        int *mag_accuracy, unsigned long long currTime, uint32_t odr);
    int (*mag_close)(float *mag_offset);
    int (*mag_update_cfg)(int *arg);
    /* prox sensor extern call */
    int (*prox_enable)(int mode, void *pdat, int len);
    int (*prox_update_cfg)(int *data);
    int (*prox_get_data)(int *value, float *raw_data);
    int (*prox_disable)(void);
    /* light sensor extern call */
    int (*light_enable)(int mode, void *pdat, int len);
    int (*light_update_cfg)(int *data);
    int (*light_get_data)(int *value, float *raw_data);
    int (*light_disable)(void);
    /* pressure sensor extern call */
    int (*pressure_enable)(int mode, void *pdat, int len);
    int (*pressure_update_cfg)(int *data);
    int (*pressure_get_data)(int *value, float *raw_data);
    int (*pressure_disable)(void);
    /* acc sensor extern call */
    int (*acc_enable)(int mode, void *pdat, int len);
    int (*acc_update_cfg)(int *data);
    int (*acc_get_data)(int *value, float *raw_data);
    int (*acc_disable)(void);
    /* gyro sensor extern call */
```



```
int (*gyro_enable)(int mode, void *pdat, int len);
int (*gyro_update_cfg)(int *data);
int (*gyro_get_data)(int *value, float *raw_data);
int (*gyro_disable)(void);

int (*extern_main)(void);
int (*get_vendor_fusion)(float *cali_acc, float *cali_gyro, float *cali_mag, uint32_t timestamp);
int (*dynamic_receive_data)(uint8_t handle, void *pdata, int len);
/* color_temp sensor extern call */
int (*color_enable)(int mode, void *pdat, int len);
int (*color_update_cfg)(int *data);
int (*color_get_data)(float *value, float *raw_data);
int (*color_disable)(void);
};
```

其中 `extern_main` 接口是在识别到有加载动态驱动时执行的第一个功能接口。如有需要，可增加一些自定义的操作，但需要注意，这个接口在每个 `sensor` 动态加载初始化时均会执行一次。`extern_main` 不论是否需要，均需要进行实例化定义，没有实际操作任务可以直接 `return`，否则动态加载会失败。其他 API 接口根据实际情况来定义，如果不需要直接写 `NULL` 即可。

3.1 磁力计 API

加速度计（Magnetic Sensor）一共有四个 API，分别在 `magnetic sensor driver` 的相应接口中被调用，这四个 API 的定义分别如下所示：

```
int (*mag_init)(int32 sampleTime_ms, float *mag_offset);
int (*mag_update)(double *mag_raw_data, double *acc, double *gyro, float *cali_mag, int *mag_accuracy,
unsigned long long currTime, uint32 odr);
int (*mag_close)(float *mag_offset);
int (*mag_update_cfg)(int *arg);
```

3.1.1 mag_init 接口

【函数功能】

使能 `magnetic sensor` 并初始化厂商算法。每次使能 `magnetic sensor` 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*mag_init)(int32 sampleTime_ms, float *mag_offset);
```

【参数说明】

参数	说明
sampleTime_ms	magnetic sensor 的采样周期。目前有四档：10 ms、20 ms、62 ms 和 200 ms。
mag_offset	上一次关闭 magnetic sensor 当校准的精度值为 3 时，所保存下来的 offset 参数。其 size 最大为 24 Byte。如果其值都为 0，则表示之前 magnetic sensor 的校准的精度值没有达到过 3，或是还没有使用过 magnetic sensor。

【返回值】

失败返回-1，正常返回 0。

3.1.2 mag_update 接口

【函数功能】

传入 acc、gyro、magnetic sensor 的 raw data，获取磁力计校准之后的数据。每次读取 raw data 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*mag_update)(double *mag_raw_data, double *acc, double *gyro, float *cali_mag, int *mag_accuracy, unsigned long long currTime, uint32 odr);
```

【参数说明】

参数	说明
mag_raw_data	mag_raw_data 是有 3 个成员的数组指针，mag_raw_data[0]、mag_raw_data[1]、mag_raw_data[2]分别存放的是 Uncalibrated mag sensor 的 X/Y/Z 轴数据，此数据的单位为 μT 。 如果调试的 sensor 比较特殊，可以使用开辟出来的 I2C 的接口，自己读取 raw data 进行相应的转换与计算，在转换完之后，需要把 μT 为单位的数据赋值给此参数，因为后续 Magnetic Uncalibrated 类型的 sensor 数据就是从此参数获取。
acc	acc 是有 3 个成员的数组指针，存放加速度的 raw data，单位为 m/s^2 。
gyro	gyro 是有 3 个成员的数组指针，存放陀螺仪的 raw data，单位为 rad/s 。
cali_mag	cali_mag 是有 3 个成员的数组指针，磁力计校准之后的 X/Y/Z 轴数据分别放到 cali_mag[0]、cali_mag[1]、cali_mag[2]，此数据以 μT 为单位。
mag_accuracy	磁力计校准后的精度值。
currTime	当前系统的时间，单位为毫秒（ms）。

参数	说明
odr	当前 magnetic sensor 的 odr。

【返回值】

失败返回-1，正常返回 0。

3.1.3 mag_close 接口

【函数功能】

关闭 magnetic sensor。每次关闭 magnetic sensor 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*mag_close)(float *mag_offset);
```

【参数说明】

参数	说明
mag_offset	获取 magnetic sensor 当前的 offset 等参数，和 init 的时候的操作相对应，最大 size 为 24 Byte。

【返回值】

失败返回-1，正常返回 0。

3.1.4 mag_update_cfg 接口

【函数功能】

传递软磁校准参数，其 size 为 36 个 int 数据，在 AP 侧配置，倘若调试时需要传递其他参数，也可以复用此接口。每次使能 magnetic sensor 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*mag_update_cfg)(int *arg);
```

【参数说明】

参数	说明
arg	magnetic sensor 软磁校准参数，其 size 为 36 个 int 数据，在 AP 侧配置。

【返回值】

失败返回-1，正常返回 0。

3.2 加速度计 API

加速度计（accelerator sensor）一共有四个 API，分别在 accelerator sensor driver 的相应接口中被调用。这四个 API 的定义分别如下所示：

```
int (*acc_enable)(int mode, void *pdat, int len);
int (*acc_update_cfg)(int *data);
int (*acc_get_data)(int *value, float *raw_data);
int (*acc_disable)(void);
```

3.2.1 acc_enable 接口

【函数功能】

使能 accelerator sensor。每次使能 accelerator sensor 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*acc_enable)(int mode, void *pdat, int len);
```

【参数说明】

参数	说明
mode	在 opcode 中配置的 position。
pdat	工厂校准中获得的三轴 offset 参数，其已经转换成 Android 坐标系，单位是 m/s ² 。
len	pdat 的长度。

【返回值】

失败返回-1，正常返回 0。

3.2.2 acc_get_data 接口

【函数功能】

获取 accelerator sensor raw data。每次读取 raw data 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*acc_get_data)(int *value, float *raw_data);
```

【参数说明】

参数	说明
value	目前传入的为 NULL。
raw_data	raw_data 是有 3 个成员的数组指针，倘若需要动态加载机制来获取 accelerator sensor raw data，那把计算好的 X/Y/Z 轴数据分别放到 raw_data[0]、raw_data[1]、raw_data[2]，此数据以 m/s ² 为单位。

【返回值】

失败返回-1，正常返回 0。

3.2.3 acc_disable 接口

【函数功能】

关闭 accelerator sensor。每次关闭 accelerator sensor 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*acc_disable)(void);
```

【参数说明】

无。

【返回值】

失败返回-1，正常返回 0。

3.2.4 acc_update_cfg 接口

【函数功能】

更新当前 accelerometer sensor 的采样周期。

【函数原型】

其函数指针声明如下所示：

```
int (*acc_update_cfg)(int *data);
```

【参数说明】

参数	说明
data	accelerometer sensor 的采样周期，目前有四档：10 ms、20 ms、60 ms、200 ms。 - data = 0：设置该 sensor 的采样周期为 10 ms。 - data = 1：设置该 sensor 的采样周期为 20 ms。 - data = 2：设置该 sensor 的采样周期为 60 ms。 - data = 3：设置该 sensor 的采样周期为 200 ms。

【返回值】

失败返回-1，正常返回 0。

3.3 陀螺仪 API

陀螺仪（gyroscope sensor）一共有四个 API，分别会在 gyroscope sensor driver 的相应接口中被调用。这四个 API 的定义分别如下所示：

```
int (*gyro_enable)(int mode, void *pdat, int len);
int (*gyro_update_cfg)(int *data);
int (*gyro_get_data)(int *value, float *raw_data);
int (*gyro_disable)(void);
```

3.3.1 gyro_enable 接口

【函数功能】

使能 gyroscope sensor。每次使能 gyroscope sensor 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*gyro_enable)(int mode, void *pdat, int len);
```

【参数说明】

参数	说明
mode	在 opcode 中配置的 position。
pdat	工厂校准中获得的三轴 offset 参数，其已经转换成 Android 坐标系，单位是 rad/s。
len	pdat 的长度。

【返回值】

失败返回-1，正常返回 0。

3.3.2 gyro_get_data 接口

【函数功能】

获取 gyroscope sensor raw data。每次读取 raw data 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*gyro_get_data)(int *value, float *raw_data);
```

【参数说明】

参数	说明
value	目前传入的为 NULL。
raw_data	raw_data 是有 3 个成员的数组指针，倘若需要动态加载机制来获取 gyroscope raw data，那把计算好的 X、Y、Z 轴数据分别放到 raw_data[0]、raw_data[1]、raw_data[2]，此数据以 rad/s 为单位。

【返回值】

失败返回-1，正常返回 0。

3.3.3 gyro_disable 接口

【函数功能】

关闭 gyroscope sensor。每次关闭 gyroscope sensor 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*gyro_disable)(void);
```

【参数说明】

无。

【返回值】

失败返回-1，正常返回 0。

3.3.4 gyro_update_cfg 接口

【函数功能】

更新当前 gyroscope sensor 的采样周期。

【函数原型】

其函数指针声明如下所示：

```
int (*gyro_update_cfg)(int *data);
```

【输入参数】

参数	说明
data	gyroscope sensor 的采样周期，目前有四档：10 ms、20 ms、60 ms、200 ms。 - data = 0：设置该 sensor 的采样周期为 10 ms。 - data = 1：设置该 sensor 的采样周期为 20 ms。 - data = 2：设置该 sensor 的采样周期为 60 ms。 - data = 3：设置该 sensor 的采样周期为 200 ms。

【返回值】

失败返回-1，正常返回 0。

3.4 压力传感器 API

压力传感器（pressure sensor）一共有四个 API，分别在 pressure sensor driver 的相应接口中被调用。这四个 API 的定义分别如下所示：

```
int (*pressure_enable)(int mode, void *pdat, int len);
int (*pressure_update_cfg)(int *data);
int (*pressure_get_data)(int *value, float *raw_data);
int (*pressure_disable)(void);
```

3.4.1 pressure_enable 接口

【函数功能】

使能 pressure sensor。每次使能 pressure sensor 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*pressure_enable)(int mode, void *pdat, int len);
```

【参数说明】

参数	说明
mode	在 opcode 中配置的 position。
pdat	工厂校准中获得的三轴 offset 参数，其已经转换成 Android 坐标系，单位是 hPa。
len	pdat 的长度。

【返回值】

失败返回-1，正常返回 0。

3.4.2 pressure_get_data 接口

【函数功能】

获取 pressure sensor raw data，每次读取 raw data 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*pressure_get_data)(int *value, float *raw_data);
```

【参数说明】

参数	说明
value	目前传入的为 NULL。
raw_data	raw_data 是有 3 个成员的数组指针，倘若需要动态加载机制来获取 pressure raw data，那把计算好的数据放到 raw_data[0]，此数据以 hPa 为单位。

【返回值】

失败返回-1，正常返回 0。

3.4.3 pressure_disable 接口

【函数功能】

关闭 pressure sensor。每次关闭 pressure sensor 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*pressure_disable)(void);
```

【参数说明】

无。

【返回值】

失败返回-1，正常返回 0。

3.4.4 pressure_update_cfg 接口

【函数功能】

更新当前 pressure sensor 的采样周期。

【函数原型】

其函数指针声明如下所示：

```
int (*pressure_update_cfg)(int *data);
```

【参数说明】

参数	说明
data	pressure sensor 的采样周期，目前有四档：10 ms、20 ms、60 ms、200 ms。 - data = 0：设置该 sensor 的采样周期为 10 ms。 - data = 1：设置该 sensor 的采样周期为 20 ms。 - data = 2：设置该 sensor 的采样周期为 60 ms。 - data = 3：设置该 sensor 的采样周期为 200 ms。

【返回值】

失败返回-1，正常返回 0。

3.5 光线传感器 API

光线传感器（light sensor）一共有四个 API，分别会在 light sensor driver 的相应接口中被调用。这四个 API 的定义分别如下所示：

```
int (*light_enable)(int mode, void *pdat, int len);
int (*light_update_cfg)(int *data);
int (*light_get_data)(int *value, float *raw_data);
int (*light_disable)(void);
```

3.5.1 light_enable 接口

【函数功能】

使能 light sensor。每次使能 light sensor 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*light_enable)(int mode, void *pdat, int len);
```

【参数说明】

参数	说明
mode	目前传入的为 NULL。
pdat	目前传入的为 NULL。
len	目前传入的为 NULL。

【返回值】

失败返回-1，正常返回 0。

3.5.2 light_get_data 接口

【函数功能】

获取 light sensor raw data。每次读取 raw data 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*light_get_data)(int *value, float *raw_data);
```

【参数说明】

参数	说明
value	目前传入的为 NULL。
raw_data	raw_data 是有 3 个成员的数组指针，倘若需要动态加载机制来获取光照强度，那把计算好的数据放到 raw_data[0]，此数据以 als 为单位。

【返回值】

失败返回-1，正常返回 0。

3.5.3 light_disable 接口

【函数功能】

关闭 light sensor。每次关闭 light sensor 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*light_disable)(void);
```

【参数说明】

无。

【返回值】

失败返回-1，正常返回 0。

3.5.4 light_update_cfg 接口

【函数功能】

更新当前 light sensor 的采样周期。

【函数原型】

其函数指针声明如下所示：

```
int (*light_update_cfg)(int *data);
```

【参数说明】

参数	说明
data	light sensor 的采样周期，目前有四档：10 ms、20 ms、60 ms、200 ms。 - data = 0：设置该 sensor 的采样周期为 10 ms。 - data = 1：设置该 sensor 的采样周期为 20 ms。 - data = 2：设置该 sensor 的采样周期为 60 ms。 - data = 3：设置该 sensor 的采样周期为 200 ms。

【返回值】

失败返回-1，正常返回 0。

3.6 距离传感器 API

距离传感器（proximity sensor）一共有四个 API，分别会在 proximity sensor driver 的相应接口中被调用。这四个 API 的定义分别如下所示：

```
int (*prox_enable)(int mode, void *pdat, int len);
int (*prox_update_cfg)(int *data);
int (*prox_get_data)(int *value, float *raw_data);
int (*prox_disable)(void);
```

3.6.1 proximity_enable 接口

【函数功能】

使能 proximity sensor。每次使能 proximity sensor 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*prox_enable)(int mode, void *pdat, int len);
```

【参数说明】

参数	说明
mode	工作模式： 1：工厂校准模式。 0：正常工作模式。
pdat	传入的是工厂校准获得的参数。 - Byte 3-Byte 0：自动校准的 calibrator(自动校准获取的底噪值)。 - Byte 7-Byte 4：手动校准(3 cm)的 calibrator(手动校准获取的接近门限值)。 - Byte 11-Byte 8：手动校准(5 cm)的 calibrator(手动校准获取的远离门限值)。 - Byte 12： 1 为自动校准(只校准底噪)，6 为手动校准(校准接近门限和远离门限)。
len	pdat 的长度。

【返回值】

失败返回-1，正常返回 0。

3.6.2 proximity_get_data 接口

【函数功能】

获取 proximity sensor raw data。每次读取 raw data 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*prox_get_data)(int *value, float *raw_data);
```

【参数说明】

参数	说明
value	目前传入的为 NULL。
raw_data	raw_data 是有 3 个成员的数组指针，需要将计算出的距离值或者接近(0)/远离(5)状态值放到 raw_data[0]，得到的距感采样值放到 raw_data[2]。

【返回值】

失败返回-1，正常返回 0。

3.6.3 proximity_disable 接口

【函数功能】

关闭 proximity sensor。每次关闭 proximity sensor 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*prox_disable)(void);
```

【参数说明】

无。

【返回值】

失败返回-1，正常返回 0。

3.6.4 proximity_update_cfg 接口

【函数功能】

更新当前 proximity sensor 的采样周期。

【函数原型】

其函数指针声明如下所示：

```
int (*prox_update_cfg)(int *data);
```

【参数说明】

参数	说明
Data	proximity sensor 的采样周期，目前有四档：10 ms、20 ms、60 ms、200 ms。 data = 0：设置该 sensor 的采样周期为 10 ms。 data = 1：设置该 sensor 的采样周期为 20 ms。 data = 2：设置该 sensor 的采样周期为 60 ms。 data = 3：设置该 sensor 的采样周期为 200 ms。

【返回值】

失败返回-1，正常返回 0。

3.7 色温传感器 API

色温传感器（color sensor）一共有四个 API，分别在 color_temp sensor driver 的相应接口中被调用。这四个 API 的定义分别如下所示：

```
int (*color_enable)(int mode, void *pdat, int len);
int (*color_update_cfg)(int *data);
int (*color_get_data)(float *value, float *raw_data);
int (*color_disable)(void);
```

3.7.1 color_enable 接口

【函数功能】

使能 color_temp sensor，每次使能 color_temp sensor 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*color_enable)(int mode, void *pdat, int len);
```

【参数说明】

参数	说明
mode	目前传入的为 NULL。
pdat	目前传入的为 NULL。
len	目前传入的为 NULL。

【返回值】

失败返回-1，正常返回 0。

3.7.2 color_update_cfg 接口

【函数功能】

更新当前 color sensor 的采样周期。

【函数原型】

其函数指针声明如下所示：

```
int (*color_update_cfg)(int *data);
```

【参数说明】

参数	说明
data	color sensor 的采样周期，目前有四档：10 ms、20 ms、60 ms、200 ms。 - data = 0：设置该 sensor 的采样周期为 10 ms。 - data = 1：设置该 sensor 的采样周期为 20 ms。 - data = 2：设置该 sensor 的采样周期为 60 ms。 - data = 3：设置该 sensor 的采样周期为 200 ms。

【返回值】

失败返回-1，正常返回 0。

3.7.3 color_get_data 接口

【函数功能】

获取 color sensor raw data，每次读取 raw data 时都会调用该接口。

【函数原型】

```
int (*color_get_data)(float *value, float *raw_data);
```

【参数说明】

参数	说明
value	计算出的 IR 通道的数值放到 value[0]中。
raw_data	raw_data 是有 3 个成员的数组指针，需要将计算出的 X、Y、Z 三个通道的数值分别放到 raw_data[0]、raw_data[1]、raw_data[2]。

【返回值】

失败返回-1，正常返回 0。

3.7.4 color_disable 接口

【函数功能】

关闭 color sensor。每次关闭 color sensor 时都会调用该接口。

【函数原型】

其函数指针声明如下所示：

```
int (*color_disable)(void);
```

【参数说明】

无

【返回值】

失败返回-1，正常返回 0。

3.8 dynamic_receive_data 接口

【函数功能】

获取从 AP 传递来的数据。

【函数原型】

```
int (*dynamic_receive_data)(uint8_t handle, void *pdata, int len);
```

【参数说明】

参数	说明
handle	sensor 类型。
pdata	从 AP 传递来的参数。
len	pdata 长度。

【返回值】

失败返回-1，正常返回 0。

【示例】

该接口需要和 AP 侧发送函数一起使用。AP 侧提供了发送函数，客户可以根据自己需求使用该发送函数。AP 侧发送函数如下：

```
static ssize_t data_to_dynamic_store(struct device *dev, struct device_attribute *attr, const char *buf, size_t count)。
```

以光线传感器从 AP 传递校准参数 light_cali 到动态加载为例介绍该接口的使用：

1. AP 侧调用 data_to_dynamic_store 节点下的 shub_send_command 函数进行数据发送：

```
uint8_t light_cali = 50;
shub_send_command(sensor, ORDER_LIGHT, AP_SEND_DATA_TO_DYNAMIC_SUBTYPE,
&light_cali, sizeof(light_cali));
```

2. 动态加载侧调用该接口进行数据解析。

4 动态加载驱动范例及编译

4.1 动态加载驱动范例

编写动态加载驱动，可以参考 vendor/sprd/modules/sensors/libsensorhub/dynamic_driver 文件夹下展锐提供的 SC9863A、UMS312、UMS512、UMS9230 平台参考驱动实现。

用户实际只需参考 vendor/sprd/modules/sensors/libsensorhub/dynamic_driver/(平台名)_dynamic_driver/dynamic_driver_(XXX).c 实现自己项目所需的 vendor/sprd/modules/sensors/libsensorhub/dynamic_driver/dynamic_driver.h 定义的接口函数即可。

- SC9863A 和 UMS312 平台参考机使用了 akm09918 地磁 sensor，使用了动态加载，详细代码参见：
 - vendor/sprd/modules/sensors/libsensorhub/dynamic_driver/sc9863a_dynamic_driver/dynamic_driver_sc9863a.c
 - vendor/sprd/modules/sensors/libsensorhub/dynamic_driver/ums312_dynamic_driver/dynamic_driver_ums312_1h10.c
- UMS512 和 UMS9230 使用了 akm09918 地磁 sensor 和 ltr578 光距感 sensor，其中对光感距感获取上报数据进行了动态加载。对距感 enable 操作使用了动态加载，详细代码参见：
 - vendor/sprd/modules/sensors/libsensorhub/dynamic_driver/ums512_dynamic_driver/dynamic_driver_ums512_1h10.c
 - vendor/sprd/modules/sensors/libsensorhub/dynamic_driver/ums9230_dynamic_driver/dynamic_driver_ums9230_1h10.c
- akm09918 地磁 sensor 驱动动态加载接口具体实现放在了 vendor/sprd/modules/sensors/libsensorhub/dynamic_driver/akm_cali/
- ltr578 光距感动态加载接口具体实现放在了 vendor/sprd/modules/sensors/libsensorhub/dynamic_driver/ltr578_driver_interface

4.2 动态加载驱动编译

请参考对应平台对应 Android 版本的 EmBitz 编译 Sensorhub 动态加载驱动介绍文档进行编译。